hashlock.

Security Audit

Monstro (DeFi)

January 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

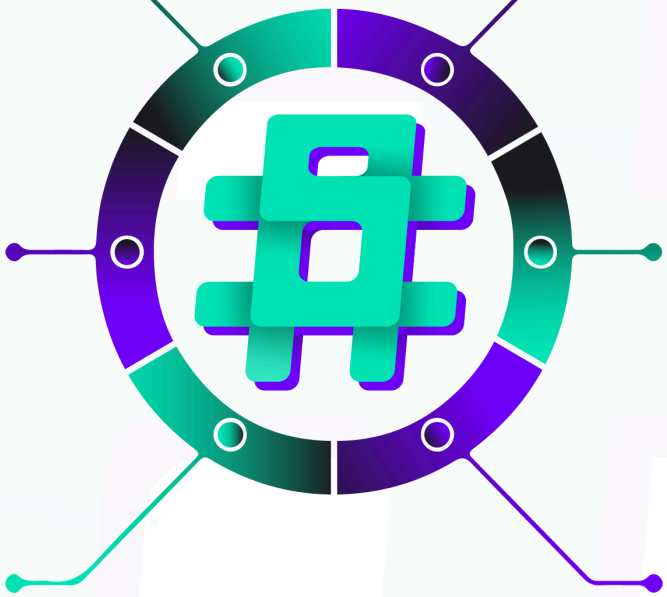
The Monstro DeFi team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Monstro DeFi is a decentralized finance ecosystem built on the Base network centered around the $MONSTRO token, which uses fixed-supply, transparent smart-contract economics to support long-term, sustainable DeFi products. The platform unifies legacy product offerings into a cohesive ecosystem with community-driven governance (DAO), predictable tokenomics, staking engines, and revenue-generating financial products designed to deliver yield and engagement for users, while emphasizing transparency and non-custodial on-chain operations.

Project Name: Monstro DeFi
Project Type: Defi
Compiler Version: ^0.8.26
Website: https://monstrodefi.com
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the Monstro DeFi project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Monstro DeFi Smart Contracts**
Platform	**Base / Solidity**
Audit Date	**January, 2026**
Contract 1	MonstroToken.sol
Contract 2	MonstroStaking.sol
Audited GitHub Commit Hash	1651bc5521ce6cb7c7d2a41ddd415d3d8a1099c8
Fix Review GitHub Commit Hash	a270edc5cef63f328ca2ea2798ee37af163b214b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High severity vulnerabilities

1 Medium severity vulnerabilities

4 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MonstroStaking.sol - Allows the operator to: - Configure protocol parameters (emissions, penalties, tiers, treasury) - Manage contract lifecycle (pause/unpause) - Administer auto-stake pools (fund pools, set Merkle roots) - Perform emergency withdrawals of excess tokens - Allows users to: - Stake, withdraw, and transfer positions - Claim and compound rewards - Activate vested auto-stake allocations - Gift stakes to other wallets	**Contract achieves this functionality.**
MonstroToken.sol - Fixed supply (400M tokens, no mint function) - Burnable (reduces total supply) - Permit support (EIP-2612 for gasless approvals) - No transfer restrictions - No blacklist functionality - No fees or taxes - Ownership renounced at deployment - No admin functions - Standard ERC20 compliance	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Monstro DeFi project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Monstro DeFi project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] MonstroStaking#stake() - New stakers can steal historical emissions

Description

The contract fails to "snapshot" the global state when a new user enters. New stakes never set `rewardPerTokenPaid` to the current `rewardPerTokenStored` which leads to stealing of rewards.

Vulnerability Details

The rewards are calculated as `(rewardPerTokenStored - userRewardPerTokenPaid) * stakeAmount`.

When a new user stakes via `stake()`, `activateAutoStake()`, or `giftStake()`, the contract sets their `amount` and `startTime`, but leaves `rewardPerTokenPaid` at default 0.

Since `rewardPerTokenStored` is a monotonically increasing global index representing all rewards ever emitted, a new user with paid = `0` is effectively treated as if they have been staking since the beginning of time. Moreover the `updateReward` modifier runs before the new stake is created but does nothing for the new user.

```
function stake(uint256 amount) external nonReentrant whenNotPaused updateReward(msg.sender) {

        StakeInfo storage userStake = stakes[msg.sender];

        if (!userStake.exists) {

            require(amount >= MIN_STAKE, "Minimum 1 token for new stake");

        } else {

            require(amount > 0, "Cannot stake 0");}



        if (!userStake.exists) {
```

```
            // New stake
            userStake.amount = amount;
            userStake.startTime = block.timestamp;
            userStake.exists = true;
            emit Staked(msg.sender, amount, amount);
        } else {
            // Adding to existing stake - time-weighted adjustment
            uint256 oldAmount = userStake.amount;
            uint256 oldStartTime = userStake.startTime;
            uint256 newTotal = oldAmount + amount;
                  // Time-weighted start time: (old_amount * old_time + new_amount *
current_time) / total
             uint256 newStartTime = (oldAmount * oldStartTime + amount * block.timestamp)
/ newTotal;
            userStake.amount = newTotal;
            userStake.startTime = newStartTime;
            emit AddedToStake(msg.sender, amount, newTotal); }
        totalStaked += amount;
        monstroToken.safeTransferFrom(msg.sender, address(this), amount);
    }
```

Since the value added to `accCollateralRewardsPerShare` is calculated by dividing against `totalSupply`, there is a considerable amount of precision loss if `collateralRewards` is small and `totalSupply` is large. This is very much the case when users are interacting with the contract frequently, causing `collateralRewards` to be small.

Proof of Concept

An example of a test that simulates reward theft is shown below.

```
function test_PoC_NewStakerStealsHistoricalEmissions() public {

        uint256 aliceStake = 1000 * 1e18;

        vm.prank(alice);

        staking.stake(aliceStake);

        vm.warp(block.timestamp + 1000);

        uint256 bobStake = 1e18;

        vm.prank(bob);

        staking.stake(bobStake);

        uint256 rptAfterBobStake = staking.rewardPerTokenStored();

        uint256 expectedStolen = (bobStake * rptAfterBobStake) / 1e18;

        uint256 bobBalanceAfterStake = token.balanceOf(bob);

        vm.prank(bob);

        staking.claimRewards();

        uint256 claimed = token.balanceOf(bob) - bobBalanceAfterStake;

        assertGt(claimed, 0, "Expected non-zero stolen rewards");

        assertEq(claimed, expectedStolen, "Claimed rewards mismatch");}
```

Impact

Direct loss of rewards from existing stakers as an attacker can drain `remainingEmissions` allocated to others.

Recommendation

We recommend explicitly initialising the user's state on creation of all stakes.

Status

Resolved

[H-02] MonstroStaking#rewardPerToken - Insolvency due to double-counting of emissions in `rewardPerToken`

Description

The contract promises more rewards than it holds, leading to a "race to claim" where the last users receive zero rewards. The `rewardPerToken` function uses `remainingEmissions` to cap the generation of new rewards, but `remainingEmissions` is only decremented when rewards are claimed. This allows the same `remainingEmissions` balance to implicitly back both "past unclaimed rewards" and "future potential rewards".

Vulnerability Details

1. `remainingEmissions` represents the physical tokens available in the contract for rewards.
2. `rewardPerToken()` calculates new rewards based on time * rate, capped by remainingEmissions.
3. As time passes, rewardPerToken increases, effectively "allocating" portions of remainingEmissions to stakers globally.

However, remainingEmissions is NOT decremented at this allocation step. It stays full and rewardPerToken continues to use the full remainingEmissions balance to justify more allocations in the next time period.

The total "Allocated Rewards" (sum of all user `earned()`) grows larger than remainingEmissions.

```
// MonstroStaking.sol:558

if (newRewards > remainingEmissions) {

    newRewards = remainingEmissions;

}

// MonstroStaking.sol:1018

remainingEmissions -= totalReward;
```

Proof of Concept

1. Emissions = 100. Stakers A and B stake.
2. Time passes. A earns 50, B earns 50. `Total Accrued` = 100.
3. `remainingEmissions` is still 100 (since no one claimed).
4. Time passes. `rewardPerToken` sees 100 available. It allows 10 more to accrue.
5. Total Accrued = 110. Real Balance = 100.
6. A claims 55. Remaining = 45.
7. B tries to claim 55. Transaction succeeds but capped at 45. B loses 10 tokens.

```
function testEmissionsInsolvency() public {

        // User 1 stakes 100 tokens

        vm.startPrank(user1);

        token.approve(address(staking), 100 * 1e18);

        staking.stake(100 * 1e18);

        vm.stopPrank();

        // User 2 stakes 100 tokens

        vm.startPrank(user2);

        token.approve(address(staking), 100 * 1e18);

        staking.stake(100 * 1e18);

        vm.stopPrank();

        // Total Staked = 200.

        // Emissions = 1 per second.

        // Split 0.5 per second each.

        // Warp 80 seconds.

        // Total emitted should be 80 tokens.

        // User 1 earned: 40. User 2 earned: 40.
```

```
        // Remaining emissions physical: 100.
        vm.warp(block.timestamp + 80);
        // Check earned
        uint256 earned1 = staking.earned(user1);
        uint256 earned2 = staking.earned(user2);
        console.log("Earned 1 (T+80):", earned1 / 1e18);
        console.log("Earned 2 (T+80):", earned2 / 1e18);
        // Check remaining emissions
        uint256 rem = staking.remainingEmissions();
        console.log("Remaining Emissions (T+80):", rem / 1e18);
        // Remaining is still 100 because nobody claimed!
        assertEq(rem, 100 * 1e18);
        // TRIGGER UPDATE REWARD at T+80
        // User 1 adds 1 token stake to trigger updateReward
        vm.startPrank(user1);
        token.approve(address(staking), 1e18);
        staking.stake(1e18);
        vm.stopPrank();
        // Now Warp 40 more seconds.
        // Total time 120 seconds.
        // Expected total emission: 100 (cap).
        // Real accrual:
        // T0-T80: 80 accrued.
        // T80-T120: 40 seconds.
        // rewardPerToken sees 100 remaining (still!).
        // It allows 40 more tokens to be accrued.
```

```
        // Total accrued = 80 + 40 = 120.
        vm.warp(block.timestamp + 40);
        earned1 = staking.earned(user1);
        earned2 = staking.earned(user2);
        console.log("Earned 1 (T+120):", earned1 / 1e18);
        console.log("Earned 2 (T+120):", earned2 / 1e18);
        uint256 totalAccrued = earned1 + earned2;
        console.log("Total Accrued:", totalAccrued / 1e18);
        // Verify Insolvency
        // Total Accrued > Total Emissions
        assertGt(totalAccrued, 100 * 1e18);
        // User 1 claims
        vm.prank(user1);
        staking.claimRewards();
        // User 2 tries to claim
        vm.prank(user2);
        staking.claimRewards();
        // Check balances
        uint256 balance1 = token.balanceOf(user1);
        uint256 balance2 = token.balanceOf(user2);
         console.log("Balance 1:", (balance1 - 900 * 1e18) / 1e18); // Subtract initial
stake remainder
        console.log("Balance 2:", (balance2 - 900 * 1e18) / 1e18);
        // One of them should get less than earned
        // Since User 1 claimed first
        // User 2 gets remainder (40).
        // But both "earned" 60.
```

```
    }
```

Impact

This leads to protocol insolvency, as the contract promises more rewards than it holds

Recommendation

We recommend the following:

- Implement a variable that tracks "rewards allocated but not claimed".
- Decrement `remainingEmission`s (or increment the new tracker variable) inside `updateReward` whenever `rewardPerTokenStored` increases.
- Ensure `rewardPerToken` only generates rewards if `remainingEmissions - rewardReserve > 0`.

Status

Resolved

Medium

[M-01] MonstroStaking#getPenaltyInfo - Understimation of penalty in the getter function

Description

There is a two-step rounding process in `getPenaltyInfo()` that first floors `penaltyBps` then floors the final penalty, whereas `withdraw()` uses a single formula in `calculatePenalty()` that avoids double rounding. This discrepancy can lead to showing a smaller penalty than what is actually enforced.

Vulnerability Details

The `withdraw()` uses `calculatePenalty()` (single formula) but the `getPenaltyInfo()` uses a two-step approach: first floors penaltyBps, then floors the final penalty again.

```
// In withdraw():

uint256 penalty = calculatePenalty(msg.sender, amount);

// In getPenaltyInfo():

penaltyBps = (MAX_PENALTY_BPS * remainingTime) / PENALTY_PERIOD;

penaltyAmount = (withdrawAmount * penaltyBps) / BPS_DENOMINATOR;<@ // Different formula
```

Impact

This inconsistency results in different penalty amount while using the getter function.

Recommendation

We recommend that `getPenaltyInfo()` calls `calculatePenalty()` directly for consistent previews.

Status

Resolved

Low

[L-01] MonstroStaking#setTierMultipliers - Unbounded tier multipliers

Description

There is a lack of a sanity check in `setTierMultipliers()` that would limit the maximum value of tier multipliers.

Vulnerability Details

It is possible to set a multiplier near `MAX_UINT` to cause a DoS via overflow.

```
function setTierMultipliers(
    uint256 _vaultBps,
    uint256 _fortressBps,
    uint256 _kingdomBps
) external onlyOwner {
    vaultMultiplierBps = _vaultBps; <@
    fortressMultiplierBps = _fortressBps;
    kingdomMultiplierBps = _kingdomBps;
    emit TierMultipliersUpdated(_vaultBps, _fortressBps, _kingdomBps);
}
```

Recommendation

We recommend adding a sanity check in the `setTierMultipliers` `function`.

Status

Resolved

[L-02] MonstroStaking#canClaimAutoStake - Broken getter function returns false incorrectly.

Description

The use of a logical `OR` in `canClaimAutoStake()` that returns false if the user has claimed either the 6-month or 12-month allocation, even if the other allocation is still available.

Vulnerability Details

A user can successfully execute the transaction and claim the second allocation even if the view function says otherwise.

```
function canClaimAutoStake(...) external view returns (bool canClaim, string memory reason) {
    // Returns false if ANY pool is claimed, even if the other is available
    if (claimed6Month[user] || claimed12Month[user]) {
        return (false, "Already claimed");
    }
    // ...
}
```

Recommendation

We recommend updating `canClaimAutoStake()` to return claimability per pool.

Status

Resolved

[L-03] MonstroStaking#calculatePenalty - Dust withdrawal can bypass the penalty due to rounding.

Description

The rounding in the penalty calculation formula that can result in a zero penalty when withdrawing tiny amounts, even if the penalty `BPS` is high. This allows users to bypass penalties for dust withdrawals, though the impact is limited due to the small amount of funds involved.

Vulnerability Details

Withdrawing tiny amount values can compute a zero penalty even at high penalty `BPS`.

```
// If amount is small, this division rounds to 0

return (amount * MAX_PENALTY_BPS * remainingTime) / (PENALTY_PERIOD * BPS_DENOMINATOR);
```

Recommendation

We recommend implementing a minimum `amount` value.

Status

Resolved

[L-04] MonstroStaking#emergencyWithdrawExcess - Function can drain protocol liabilities instead of just excess funds.

Description

The `emergencyWithdrawExcess` function allows the owner to withdraw any tokens held by the contract that are not actively staked by users (totalStaked). However, the contract holds significant funds that are liabilities even if they are not user stakes, such as remainingEmissions (rewards owed to future stakers), `unassigned6MonthPool` / `unassigned12MonthPool` (vesting allocations), and `pendingTreasuryPayments` (fees owed to the treasury). By treating `contractBalance - totalStaked` as "excess", the admin can drain these critical reserves, causing immediate insolvency.

Vulnerability Details

This can lead to direct protocol insolvency. If the admin calls this function (maliciously or accidentally believing they are only taking "extra" tokens), the contract will lose the ability to pay out:

- Accumulated rewards (remainingEmissions).
- Auto-stake claims (unassigned pools).
- Treasury fees (pendingTreasuryPayments).

```
function emergencyWithdrawExcess(address to) external onlyOwner {

    // ...

    uint256 contractBalance = monstroToken.balanceOf(address(this));

    require(contractBalance > totalStaked, "No excess tokens");

    // BUG: Treats everything except principal as "excess", ignoring liabilities

    uint256 excessAmount = contractBalance - totalStaked;

    // ...

}
```

Recommendation

We recommend updating the function to explicitly calculate all liabilities before determining the excess amount.

Status

Resolved

Centralisation

The Monstro DeFi project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

The token contract is renounced and has no owner/admin functions and the staking contract shall be owned by the Monstro DAO multisig.

Conclusion

After Hashlock's analysis, the Monstro DeFi project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au